

Rapport du projet logique

Aichaoui Wassim et Deruwez Laura

Introduction:

Ce projet consiste en l'implémentation de multiplication entre deux matrices, et de recherches de solution à des multiplications de matrices. Il est réalisé en Prolog. Les matrices concernées sont en base binaire et ternaire.

1. Partie 1 : multiplication de matrices binaires

Pour cette première partie, pour manipuler les valeurs binaires, nous implantons les règles d'opérations de l'énoncé grâce à des faits :

```
addition(o, o, o).
addition(u, o, u).
addition(o, u, u).
addition(u, u, o).
multiplication(o, o, o).
multiplication(u, o, o).
multiplication(o, u, o).
multiplication(u, u, u).
```

Pour continuer et manipuler des matrices avec des valeurs binaires et se rapprocher de la multiplication, on commence par l'addition des vecteurs lignes. Elle est réalisée élément par élément, et récursivement.

```
addition_vecteur_ligne([], [], []).
addition_vecteur_ligne([X|L1], [Y|L2], [Z|L3]) :-
    addition(X, Y, Z),
    addition_vecteur_ligne(L1, L2, L3).
```

La première ligne est le cas de base avec aucun élément.

La seconde ligne contient l'appel récursif.

Pour pouvoir effectuer la multiplication des matrices et créer le résultat, il faut un prédicat qui génère un vecteur nul avec que des o.

Pour éviter une récursion infinie, nous vérifions N avec $N > 0$ et $N1$ is $N - 1$. Cela implique que lors de l'appel, N soit initialisé. Mais ce n'est pas un problème car pour la suite nous l'utiliserons avec N déjà initialisé.

```
vecteur_nul(0, []).
vecteur_nul(N, [o|L]) :-
    N > 0,
    N1 is N - 1,
    vecteur_nul(N1, L).
```

Nous passons maintenant à la partie multiplication, très importante pour pouvoir ensuite deviner les solutions.

Ici nous avons les deux cas de base : - soit le dernier élément de V (donc le seul élément dans le cas de base) est un u et alors on recopie la dernière ligne de la matrice concernée (la seule dans le cas de base) Exemple : $[u] \times [[u,o,u]] = [u,o,u]$ - soit c'est un o et on génère un vecteur nul pour la base de la réponse Exemple : $[o] \times [[u,o,u]] = [o,o,o]$	<pre>mult_vect_mat([u], [L], L). mult_vect_mat([o], [L], Lo) :- length(L, X), Q vecteur_nul(X, Lo), !.</pre>
---	--

Ensuite nous avons deux cas différent pour l'appel récursif :	<pre>mult_vect_mat([o V], [_ M], R) :- mult_vect_mat(V, M, R). mult_vect_mat([u V], [L M], R2) :- mult_vect_mat(V, M, R1), addition_vecteur_ligne(L, R1, R2).</pre> - On l'appelle avec une ligne multipliée par o, donc le résultat reste le même - On l'appelle avec une ligne multipliée par u, donc le résultat devient l'ancien résultat auquel on ajoute la ligne concernée.
--	---

Pour pouvoir générer des solutions, il fallait être capable de créer tous les vecteurs binaires possibles de longueur N (pour ensuite tester le résultat de la multiplication). Ce qui est fait récursivement ici. Comme au-dessus pour vecteur_nul, il faut que N soit initialisé à l'appel.	<pre>vecteur_binaire([], 0). vecteur_binaire([X V], N) :- N>0, N1 is N-1, vecteur_binaire(V, N1), valeur_binaire(X).</pre>
---	---

Une fois que nous avons les prédicats pour générer les vecteurs, et la multiplication entre un vecteur et une matrice, nous pouvons passer à générer des solutions. solution/2 unifie à V le vecteur par lequel il faudrait multiplier la matrice pour obtenir un vecteur nul.

Avec les deux <code>length</code>, on récupère le nombre de lignes (Nl) et le nombre de colonnes (Nc) de la matrice M. On peut donc créer le vecteur nul de la solution (Zero) et des vecteurs binaires (V) et appeler <code>mul_vect_mat</code> avec ces deux vecteurs. La multiplication sera vraie si V est la solution, donc nous aurons bien la solution dans V.	<pre>solution(M, V) :- length(M, Nl), M = [Pl _], length(Pl, Nc), vecteur_nul(Nc, Zero), vecteur_binaire(V, Nl), mult_vect_mat(V, M, Zero).</pre>
--	---

Pour la suite, il faut pouvoir avoir une solution avec un nombre défini de u. Nous avons donc créé un prédicat `compte_vrai` qui nous donne le nombre de u dans le vecteur.

Le cas de base : il y a 0 u dans une liste vide. Puis on a deux cas d'appel récursif : - si on ajoute u, on ajoute 1 à N - si on ajoute o, N reste identique	<pre>compte_vrai([], 0). compte_vrai([u L], N) :- compte_vrai(L, N1), N is N1 + 1. compte_vrai([o L], N) :- compte_vrai(L, N).</pre>
---	--

Une fois que nous avons cette fonction, nous pouvons donc créer `solution_binaire(M, V, Nombre)`. Contrairement à l'énoncé, `Nombre` ici est une variable et non une constante (nombre dans l'énoncé). C'est plus logique qu'une constante si nous voulons pouvoir l'utiliser avec des entiers que nous fournissons. Et puis cela ne fonctionne pas avec `nombre` avec une minuscule car il faut pouvoir faire des additions avec (dans `compte_vrai`) et `nombre` n'est pas un entier mais une constante, alors que `Nombre` peut être un entier.

Pour ces raisons, notre `Nombre` est une variable et non une constante. Cela dit, cette différence mise à part, le nom de la fonction est le même.

Cette justification compte pour `solution_binaire(M, V, Nombre)` et `solution_binaire(M, V, Y, Nombre)`.

`Solution_binaire(M, V, Nombre)` renvoie donc les solutions V avec `Nombre` u.

On récupère d'abord le nombre de lignes Nl et le nombre de colonnes Nc pour générer les vecteurs comme précédemment dans solution, ici en plus nous avons une clause supplémentaire compte_vrai(V, Nombre) qui s'assure que V a bien le bon nombre de u.	<pre> solution_binaire(M, V, Nombre) :- length(M, Nl), M = [Pl _], length(Pl, Nc), vecteur_nul(Nc, Zero), vecteur_binaire(V,Nl), compte_vrai(V, Nombre), mult_vect_mat(V, M, Zero). </pre>
--	--

On peut finalement faire solution_binaire(M, V, Y, Nombre) avec V le vecteur avec Nombre, qui multiplié à M donne Y.

Ici, il suffit de récupérer le nombre de lignes pour générer un vecteur binaire V, puisqu'on ne doit pas générer le résultat de la multiplication attendu qui est Y. Sinon le fonctionnement est le même qu'au-dessus.	<pre> solution_binaire(M, V, Y, Nombre) :- length(M, Nl), vecteur_binaire(V, Nl), compte_vrai(V, Nombre), mult_vect_mat(V, M, Y). </pre>
--	--

Nous avons donc testé notre programme avec les matrices et les Y donnés en .txt sur Community.

Elles ont été enregistrées sous la forme m2_facile(MATRICE), et y2_facile(Y), pour pouvoir être unifiées dans des commandes dans le terminal.

Voici la commande rentrée dans le terminal et les 3 premières réponses données.

Matrice facile	?- m2_facile(M), y2_facile(Y), solution_binaire(M, V, Y, 10).
	<pre> [u,u,u,o,u,o,u,o,o,u,o,o,o,u,u,u,u,o,o,o,o,o,o,o,o] [u,o,u,o,o,u,o,u,u,u,o,u,u,o,o,u,o,o,o,o,o,o,o,o] [o,o,u,u,u,o,o,u,u,o,u,u,o,o,u,u,o,u,o,o,o,o,o,o,o] </pre>
Matrice moyenne	?- m2_moyen(M), y2_moyen(Y), solution_binaire(M, V, Y, 7).
	<pre> [u,o,o,o,o,u,u,o,o,o,o,u,u,o,u,u,o,o,o,o,o,o,o,o] [u,o,u,u,o,o,o,o,u,o,o,o,u,o,o,u,u,o,o,o,o,o,o,o] [u,u,o,o,o,o,o,o,u,u,o,u,o,o,u,u,o,o,o,o,o,o,o,o] </pre>
Matrice difficile	?- m2_dur(M), y2_dur(Y), solution_binaire(M, V, Y, 6).
	<pre> [u,o,o,u,o,o,o,o,o,o,o,o,u,o,u,o,o,u,o,o,u,o,o,o,o] [o,u,o,u,o,o,o,o,u,o,o,o,o,o,u,o,o,o,u,o,u,o,o,o,o] [o,o,u,o,o,o,u,o,o,u,o,o,o,o,o,o,o,u,u,o,u,o,o,o,o] </pre>

Le principal problème de ce programme est sa complexité. Il génère et teste tous les vecteurs existants de la bonne taille. Pour une taille N , il y a 2^N vecteurs possible, donc plus N augmente, plus il met de temps à trouver. De plus, il ne faut pas oublier qu'il vérifie aussi avec la multiplication. La complexité est donc désastreuse.

Par exemple, la génération des trois solutions avec la matrice facile a pris 518 ms et celle avec la matrice difficile 3204 ms.

2. Partie 2 : Multiplication de matrices ternaires

La deuxième partie du projet suit exactement la même logique, mais en introduisant une troisième valeur $d = 2$.

On commence par déclarer les valeurs ternaires.	<code>valeur_ternaire(o).</code> <code>valeur_ternaire(u).</code> <code>valeur_ternaire(d).</code>
Puis rajouter les additions ternaires	<code>addition3(o, o, o).</code> <code>addition3(u, o, u).</code> <code>addition3(o, u, u).</code> <code>addition3(d, o, d).</code> <code>addition3(o, d, d).</code> <code>addition3(u, u, d).</code> <code>addition3(d, u, o).</code> <code>addition3(u, d, o).</code> <code>addition3(d, d, u).</code>
Et pour finir les multiplications ternaires	<code>multiplication3(o, o, o).</code> <code>multiplication3(o, u, o).</code> <code>multiplication3(u, o, o).</code> <code>multiplication3(o, d, o).</code> <code>multiplication3(d, o, o).</code> <code>multiplication3(u, u, u).</code> <code>multiplication3(u, d, d).</code> <code>multiplication3(d, u, d).</code> <code>multiplication3(d, d, u).</code>

Le prédicat `addition_vecteur_ligne3/3` garde la même logique mais utilise `addition3` et pas `addition` qui était l'addition binaire.

Ensuite, on avait défini le prédicat `vecteur_nul`, il n'est plus utile dans la multiplication mais le sera dans la recherche de solutions, que l'on abordera plus tard.

Pour la multiplication, plutôt que de faire un cas où la valeur est 0 et on ne fait rien, et un cas où la valeur est u et on ajoute la ligne, on fait un prédicat `mult_scalaire3` qui multiplie un vecteur ligne par un scalaire. Le résultat est le 3e paramètre.

On a un cas de base avec les deux listes vides. Puis un appel récursif qui traite un par un les vecteurs, ajoutant au résultat en première place le premier élément du vecteur multiplié par le scalaire.	<pre> mult_scalaire3(_, [], []). mult_scalaire3(S, [X L], [Z R]) :- multiplication3(S, X, Z), mult_scalaire3(S, L, R). </pre>
--	---

Ensuite, la multiplication utilise les fonctions définies précédemment :

Cas de base avec un vecteur à un seul élément et une matrice à une seule ligne. Le résultat est donc la multiplication de la ligne de la matrice par le scalaire X (seul élément du vecteur).	<pre> mult_vect_mat3([X], [L], R) :- mult_scalaire3(X, L, R). </pre>
Ici on a deux appels récursifs : - le premier gère le cas où le scalaire est 0, on laisse le résultat comme avant - le second c'est dans le cas où c'est u ou d, et il ajoute le résultat de la multiplication avec le scalaire de la ligne de la matrice au résultat final.	<pre> mult_vect_mat3([0 V], [_ M], R) :- mult_vect_mat3(V, M, R). mult_vect_mat3([X V], [L M], R2) :- X \= 0, mult_vect_mat3(V, M, R1), mult_scalaire3(X, L, LX), addition_vecteur_ligne3(LX, R1, R2). </pre>

C'était un choix lors de la création de la fonction, il y aurait pu avoir un seul appel récursif qui gère 0, u, et d, en faisant la multiplication par le scalaire et en l'ajoutant au résultat. Cependant, dans le cas de 0, cela fait donc une multiplication par un scalaire en plus pour ajouter un vecteur nul qui au final ne changera pas le résultat. Nous avons donc décidé de le traiter en deux cas séparés pour éviter ces opérations inutiles.

Pour cette partie, nous avons décidé de faire comme pour la partie 1 :

- un prédicat `solution_ternaire(M, V)` avec `V` qui est la solution pour que la multiplication $V \times M$ donne un vecteur nul.
- un prédicat `solution_ternaire(M, V, Nombre)`, qui donne le même résultat mais avec `Nombre` éléments différents de 0 dans la solution
- un prédicat `solution_ternaire(M, V, Y, Nombre)`, qui est le prédicat demandé dans l'énoncé

Dans ce but, nous utiliserons les prédicats :

Vecteur_ternaire(V, N) qui a le même fonctionnement que vecteur_binaire(V, N).	<pre> vecteur_ternaire([], 0). vecteur_ternaire([X L], N) :- N > 0, N1 is N - 1, vecteur_ternaire(L, N1), valeur_ternaire(X). </pre>
compte_non_nul3(V, N) qui renvoie le nombre de valeurs différentes de 0. Elle a un cas de base avec la liste nulle qui ne contient aucune valeur différente de 0. Et deux appels récursifs selon si la valeur ajoutée à la liste est 0 ou non.	<pre> compte_non_nul3([], 0). compte_non_nul3([o L], N) :- compte_non_nul3(L, N). compte_non_nul3([X L], N) :- compte_non_nul3(L, N1), valeur_ternaire(X), X /= 0, N is N1 + 1. </pre>

Pour les trois fonctions de solution, le principe est le même que pour la partie binaire, en utilisant les prédicats adaptés (ex : `vecteur_ternaire`, `mult_vect_mat3`, `compte_non_nul3`).

Nous avons donc testé notre programme avec les matrices et les `Y` donnés en `.txt` sur Community.

Elles ont été enregistrées sous la forme `m3_facile(MATRICE)`, et `y3_facile(Y)`, afin d'être unifiées dans des commandes dans le terminal.

Voici la commande rentrée dans le terminal et les 3 premières réponses données.

Matrice facile	?- m3_facile(M), y3_facile(Y), solution_ternaire(M, V, Y, 10).
	[d,d,o,o,d,d,d,u,d,u,u,d,o,o,o,o,o,o] (1076098 ms)
Matrice moyenne	?- m3_moyen(M), y3_moyen(Y), solution_ternaire(M, V, Y, 8).
	[u,u,o,u,d,o,d,d,o,d,o,u,o,o,o,o,o,o] (710195 ms)

Matrice difficile	?- m3_dur(M), y3_dur(Y), solution_ternaire(M, V, Y, 5).
-------------------	---

Dans le cas ternaire, comme il y a plus de possibilités pour des vecteurs de longueur N, la complexité est encore pire. Il y a 3^N vecteurs possibles, qu'il teste tous.

Les résultats prenant beaucoup trop de temps à arriver, il n'est pas possible de tous les renseigner ici. Cependant, nous verrons dans les tests que le programme fonctionne parfaitement avec des matrices plus petites.

Cela montre les limites de la recherche par force brute en Prolog.

3. Tests et exemples d'utilisation du programme

Tests de la partie binaire

Commande dans le terminal	Résultat obtenu
vecteur_nul	
?- vecteur_nul(0, L).	L = []
?- vecteur_nul(3, L)	L = [o,o,o]
vecteur	
?- vecteur(V, 0).	V = []
?- vecteur(V, 1).	V = [o] ; V = [u]
?- vecteur(V, 2).	V = [o,o] ; V = [o,u] ; V = [u,o] ; V = [u,u]
addition_vecteur_ligne	
?- addition_vecteur_ligne([o], [u], Z).	Z = [u]
?- addition_vecteur_ligne([u], [u], Z).	Z = [o]
?- addition_vecteur_ligne([o,u,u], [u,u,o], Z)	Z = [u,o,u]
?- addition_vecteur_ligne([u,u], [u,u], Z).	Z = [o,o]
mult_vect_mat	
?- mult_vect_mat([u], [[o,u,o]], R).	R = [o,u,o]
?- mult_vect_mat([o], [[o,u,o]], R).	R = [o,o,o]
?- mult_vect_mat([u,o], [[o,u],[u,o]], R).	R = [o,u]

?- mult_vect_mat([u,u], [[o,u],[u,o]], R).	R = [u,u]
?- mult_vect_mat([u,o,u], [[o,u],[u,o],[u,u]], R).	R = [u,o]
?- mult_vect_mat([u,u,u], [[o,u],[u,o],[u,u]], R).	R = [o,o]

solution(M, V)	
?- solution([u,o],[o,u], V).	V = [o,o]
?- solution([o,o],[o,o], V).	V = [o,o] ; V = [o,u] ; V = [u,o] ; V = [u,u]
?- solution([u,u],[u,u], V).	V = [o,o] ; V = [u,u]
?- solution([u,u,o],[u,u,o], V).	V = [o,o] ; V = [u,u]
compte_vrai(V, N)	
?- compte_vrai([], N).	N = 0
?- compte_vrai([u,o,u], N).	N = 2
?- compte_vrai([o,o,o], N).	N = 0
?- compte_vrai([u,u,u], N).	N = 3
solution_binaire(M, V, Nombre).	
?- solution_binaire([o,o],[o,o], V, Nombre).	V = [o,o], Nombre = 0 ; V = [o,u], Nombre = 1 ; V = [u,o], Nombre = 1 ; V = [u,u], Nombre = 2
?- solution_binaire([u,o],[o,u], V, Nombre).	V = [o,o], Nombre = 0
?- solution_binaire([u,u],[u,u], V, Nombre).	V = [o,o], Nombre = 0 ; V = [u,u], Nombre = 2
solution_binaire(M, V, Y, Nombre).	
?- solution_binaire([u,o],[o,u], V, [o,u], Nombre).	V=[o,u], Nombre=1
?- solution_binaire([u,u],[u,u], V, [u,u], 1).	V=[u,o] ; V=[o,u]

?- solution_binaire([[u,u,o],[o,u,u]], V, [u,o,u], Nombre).	V=[u,u], Nombre=2
---	-------------------

Tests de la partie ternaire

addition_vecteur_ligne3	
?- addition_vecteur_ligne3([], [], R).	R = []
?- addition_vecteur_ligne3([u,d,o], [o,o,o], R).	R = [u,d,o]
?- addition_vecteur_ligne3([o,o,o], [u,d,o], R).	R = [u,d,o]
?- addition_vecteur_ligne3([u,d,u], [d,u,d], R).	R = [o,o,o]
?- addition_vecteur_ligne3([u,u,d], [u,d,u], R).	R = [d,o,o]
?- addition_vecteur_ligne3([d,u,o], [d,d,u], R).	R = [u,o,u]
mult_scalaire3	
?- mult_scalaire3(o, [u,d,u], R).	R = [o,o,o]
?- mult_scalaire3(u, [u,d,o,u], R).	R = [u,d,o,u]
?- mult_scalaire3(d, [u,d,o,u], R).	R = [d,u,o,d]
?- mult_scalaire3(d, [], R).	R = []
mult_vect_mat3	
?- mult_vect_mat3([u,o], [[u,o],[o,u]], R).	R = [u,o]
?- mult_vect_mat3([o,d], [[u,o],[o,u]], R).	R = [o,d]
?- mult_vect_mat3([o,o], [[u,d],[d,u]], R).	R = [o,o]
?- mult_vect_mat3([u,u], [[u,d],[d,u]], R).	R = [o,o]
?- mult_vect_mat3([d,d], [[u,d],[d,u]], R).	R = [o,o]
?- mult_vect_mat3([u,o], [[u,d],[d,u]], R).	R = [u,d]
vecteur_ternaire	
?- vecteur_ternaire(V, 0).	V = []
?- vecteur_ternaire([o,u,d], 3).	true
?- vecteur_ternaire([o,x,d], 3).	false

?- vecteur_ternaire([o,u], 3).	false
compte_non_nul3	
?- compte_non_nul3([], N).	N = 0
?- compte_non_nul3([o,o,o,o], N).	N = 0
?- compte_non_nul3([o,u,o], N).	N = 1
?- compte_non_nul3([u,d,u], N).	N = 3
?- compte_non_nul3([u,o,d], N).	N = 2
solution_ternaire(M, V)	
?- solution_ternaire([[u,d],[d,u]], [o,o]).	true
?- solution_ternaire([[u,d],[d,u]], [u,u]).	true
?- solution_ternaire([[u,d],[d,u]], [d,d]).	true
?- solution_ternaire([[u,d],[d,u]], [u,d]).	false
solution_ternaire(M, V, Nombre)	
?- solution_ternaire([[u,d],[d,u]], V, 0).	V = [o,o]
?- solution_ternaire([[u,d],[d,u]], V, 2).	V = [u, u]
?- solution_ternaire([[u,d],[d,u]], V, 2).	V = [d, d]

Conclusion :

Ce projet était intéressant, car il nous a fait réfléchir à la multiplication de matrices, et comment l'implémenter en prolog. Une des contraintes était aussi que le programme puisse générer les solutions, ce qui a forcé à voir les choses autrement dans la récursion. Cependant, il reste une frustration quant à l'utilisation de Nombre à la place de nombre car cela ne fonctionnait pas avec nombre (affichait no). Ou plutôt, une réelle interrogation sur comment il aurait fallu procéder avec une constante nombre, qui n'est finalement jamais associée à une valeur.